

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME:** Database Management Systems

**COURSE CODE:** CSA05

---

### COURSE OUTCOME COVERED

**CO2:** *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

---

| QUESTIONS |                                                                                                                                                                                                |                |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| Q.No      | Question                                                                                                                                                                                       | Bloom's Level  |
| 1         | Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.<br>Bloom's Level: BL3 – Apply             | BL4 – Analyze  |
| 2         | Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.<br>Bloom's Level: BL4 – Analyze         | BL5 – Evaluate |
| 3         | Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates.<br>Bloom's Level: BL6 – Create | BL6 – Create   |
| 4         | Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution.<br>Bloom's Level: BL4 – Analyze                                                    | BL6 – Create   |
| 5         | Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency.<br>Bloom's Level: BL5 – Evaluate                                            | BL4 – Analyze  |
| 6         | Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.<br>Bloom's Level: BL6 – Create                         | BL5 – Evaluate |
| 7         | Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.<br>Bloom's Level: BL4 – Analyze           | BL4 – Analyze  |

|           |                                                                                                                                                                                         |                |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <b>8</b>  | Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements.<br>Bloom's Level: BL5 – Evaluate       | BL5 – Evaluate |
| <b>9</b>  | Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem.<br>Bloom's Level: BL6 – Create                                           | BL6 – Create   |
| <b>10</b> | Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning.<br>Bloom's Level: BL5 – Evaluate | BL5 – Evaluate |

---

**1. Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.**

### **Step 1: Create Tables**

```
CREATE TABLE DEPARTMENT (  
    DepartmentID INT PRIMARY KEY,  
    DepartmentName VARCHAR(30)  
);  
CREATE TABLE STUDENT (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(30),  
    DepartmentID INT  
);  
CREATE TABLE MARKS (  
    StudentID INT,  
    Subject VARCHAR(30),  
    Marks INT  
);
```

### **Step 2: Insert Sample Data**

```
INSERT INTO DEPARTMENT VALUES  
(1,'AIML'),  
(2,'CSE'),  
(3,'ECE');  
INSERT INTO STUDENT VALUES  
(101,'Durga',1),  
(102,'Rahul',2),  
(103,'Priya',1),  
(104,'Arjun',3);  
INSERT INTO MARKS VALUES  
(101,'DBMS',90),  
(102,'DBMS',75),  
(103,'DBMS',88),  
(104,'DBMS',65);
```

### **Query 1: INNER JOIN**

```
SELECT S.Name, D.DepartmentName  
FROM STUDENT S  
INNER JOIN DEPARTMENT D  
ON S.DepartmentID = D.DepartmentID;
```

Output:

| Name  | DepartmentName |
|-------|----------------|
| Durga | AIML           |
| Rahul | CSE            |
| Priya | AIML           |
| Arjun | ECE            |

### Query 2: Multiple JOIN

```
SELECT S.Name, D.DepartmentName, M.Marks
FROM STUDENT S
INNER JOIN DEPARTMENT D
ON S.DepartmentID = D.DepartmentID
INNER JOIN MARKS M
ON S.StudentID = M.StudentID;
```

| Name  | DepartmentName | Marks |
|-------|----------------|-------|
| Durga | AIML           | 90    |
| Rahul | CSE            | 75    |
| Priya | AIML           | 88    |
| Arjun | ECE            | 65    |

### Query 3: Students Above Average Marks (Subquery)

```
SELECT Name
FROM STUDENT
WHERE StudentID IN
(
SELECT StudentID
FROM MARKS
WHERE Marks >
(
SELECT AVG(Marks)
FROM MARKS

```

```
)  
);
```

```
+-----+  
| Name  |  
+-----+  
| Durga |  
| Priya |  
+-----+
```

#### Query 4: Department with More Than One Student

```
SELECT DepartmentName  
FROM DEPARTMENT  
WHERE DepartmentID IN  
(  
SELECT DepartmentID  
FROM STUDENT  
GROUP BY DepartmentID  
HAVING COUNT(*) > 1  
);
```

```
+-----+  
| DepartmentName |  
+-----+  
| AIML           |  
+-----+
```

#### Query 5: Student with Highest Marks

```
SELECT Name  
FROM STUDENT  
WHERE StudentID =  
(  
SELECT StudentID  
FROM MARKS  
WHERE Marks =  
(  
SELECT MAX(Marks)  
FROM MARKS  
)  
);
```

| Name  |
|-------|
| Durga |

**2.Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.**

### Algebraic Notations:

Retrieve names of students belonging to the AIML department:

```
 $\pi$  Name ( $\sigma$  DepartmentName='AIML'
(STUDENT  $\bowtie$  STUDENT.DepartmentID = DEPARTMENT.DepartmentID DEPARTMENT))
```

Retrieve students who scored more than 80 marks:

```
 $\pi$ Name(STUDENT $\bowtie$ STUDENT.StudentID=MARKS.StudentID( $\sigma$ Marks>80(MARKS)))
```

**3.Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates.**

### Step 1: Create Tables

```
CREATE TABLE STUDENT (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(30),
    Department VARCHAR(20),
    Age INT
);
CREATE TABLE COURSE (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(30),
    Faculty VARCHAR(30)
);
CREATE TABLE ENROLLMENT (
    StudentID INT,
    CourseID INT,
    Marks INT,
    PRIMARY KEY (StudentID, CourseID),
    FOREIGN KEY (StudentID) REFERENCES STUDENT(StudentID),
    FOREIGN KEY (CourseID) REFERENCES COURSE(CourseID)
);
```

## Step 2: Insert Sample Data

INSERT INTO STUDENT VALUES

(101,'Durga','AIML',20),

(102,'Rahul','CSE',21),

(103,'Priya','AIML',20);

INSERT INTO COURSE VALUES

(1,'DBMS','Ramesh'),

(2,'Python','Suresh');

INSERT INTO ENROLLMENT VALUES

(101,1,90),

(102,1,75),

(103,2,88);

## Query 1: Retrieve all student details

SELECT \* FROM STUDENT;

Output:

| StudentID | Name  | Department | Age |
|-----------|-------|------------|-----|
| 101       | Durga | AIML       | 20  |
| 102       | Rahul | CSE        | 21  |
| 103       | Priya | AIML       | 20  |

## Query 2: Retrieve student names with their course names

SELECT S.Name, C.CourseName  
FROM STUDENT S  
JOIN ENROLLMENT E ON S.StudentID = E.StudentID  
JOIN COURSE C ON E.CourseID = C.CourseID;

Output:

| StudentID | Name  | Department | Age |
|-----------|-------|------------|-----|
| 101       | Durga | AIML       | 20  |
| 102       | Rahul | CSE        | 21  |
| 103       | Priya | AIML       | 20  |

  

| Name  | CourseName |
|-------|------------|
| Durga | DBMS       |
| Rahul | DBMS       |
| Priya | Python     |

### Query 3: Retrieve students scoring more than 80 marks

```
SELECT S.Name, E.Marks
FROM STUDENT S
JOIN ENROLLMENT E
ON S.StudentID = E.StudentID
WHERE E.Marks > 80;
```

Output:

```
+-----+-----+-----+-----+
| StudentID | Name | Department | Age |
+-----+-----+-----+-----+
|      101 | Durga | AIML      | 20 |
|      102 | Rahul | CSE       | 21 |
|      103 | Priya | AIML      | 20 |
+-----+-----+-----+-----+

+-----+-----+
| Name | CourseName |
+-----+-----+
| Durga | DBMS      |
| Rahul | DBMS      |
| Priya | Python    |
+-----+-----+

+-----+-----+
| Name | Marks |
+-----+-----+
| Durga | 95 |
| Priya | 88 |
+-----+-----+
```

**4. Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution.**

#### 1. Create STUDENT Table

```
CREATE TABLE STUDENT (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(30),
    Department VARCHAR(20)
);
```

#### 2. Create MARKS Table

```
CREATE TABLE MARKS (
    StudentID INT,
    Marks INT
);
```



### 3. Insert Sample Data into STUDENT

INSERT INTO STUDENT VALUES

(101,'Durga','AIML'),

(102,'Rahul','CSE'),

(103,'Priya','AIML'),

(104,'Arjun','ECE');

### 4. Insert Sample Data into MARKS

INSERT INTO MARKS VALUES

(101,90),

(102,75),

(103,88),

(104,65);

### Query 1: Verify the Data

SELECT \* FROM STUDENT;

Output:

| StudentID | Name  | Department |
|-----------|-------|------------|
| 101       | Durga | AIML       |
| 102       | Rahul | CSE        |
| 103       | Priya | AIML       |
| 104       | Arjun | ECE        |

### Query 2: Select marks

SELECT \* FROM MARKS;

| StudentID | Marks |
|-----------|-------|
| 101       | 90    |
| 102       | 75    |
| 103       | 88    |
| 104       | 65    |

### Query 3: Calculate the Average Marks

```
SELECT AVG(Marks)
FROM MARKS;
```

```
+-----+
| AVG(Marks) |
+-----+
|    79.5000 |
+-----+
```

### Query 4: Retrieve Student IDs with Marks Greater than the Average

```
SELECT StudentID
FROM MARKS
WHERE Marks >
(
    SELECT AVG(Marks)
    FROM MARKS
);
```

```
+-----+
| AVG(Marks) |
+-----+
|    79.5000 |
+-----+
+-----+
| StudentID  |
+-----+
|         101 |
|         103 |
+-----+
```

**5. Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency.**

### Create Student Table:

```
CREATE TABLE STUDENT (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(30),
    Department VARCHAR(20)
);
INSERT INTO STUDENT VALUES
(101,'Durga','AIML'),
(102,'Rahul','CSE'),
(103,'Priya','AIML'),
(104,'Arjun','ECE');
```

**MARKS:**

```
CREATE TABLE MARKS (  
    StudentID INT,  
    Marks INT  
);  
INSERT INTO MARKS VALUES  
(101,90),  
(102,75),  
(103,88),  
(104,65);
```

**Query 1: Using JOIN**

```
SELECT S.Name, M.Marks  
FROM STUDENT S  
JOIN MARKS M  
ON S.StudentID = M.StudentID  
WHERE M.Marks > 80;
```

---

Output:

|   |       |   |       |   |
|---|-------|---|-------|---|
| + | ----- | + | ----- | + |
|   | Name  |   | Marks |   |
| + | ----- | + | ----- | + |
|   | Durga |   | 90    |   |
|   | Priya |   | 88    |   |
| + | ----- | + | ----- | + |

**Query 2: Retrieve Students Scoring Above the Average Marks**

```
SELECT S.Name, M.Marks  
FROM STUDENT S  
JOIN MARKS M  
ON S.StudentID = M.StudentID  
WHERE M.Marks >  
(  
    SELECT AVG(Marks)  
    FROM MARKS  
);
```

Output:

| Name  | Marks |
|-------|-------|
| Durga | 90    |
| Priya | 88    |

  

| Name  | Marks |
|-------|-------|
| Durga | 90    |

### Query 3: Retrieve the Student(s) with the Highest Marks

```
SELECT S.Name, M.Marks
FROM STUDENT S
JOIN MARKS M
ON S.StudentID = M.StudentID
WHERE M.Marks =
(
    SELECT MAX(Marks)
    FROM MARKS
);
```

Output:

| Name  | Marks |
|-------|-------|
| Durga | 90    |
| Priya | 88    |

  

| Name  | Marks |
|-------|-------|
| Durga | 90    |

### Query 4: Retrieve Students from the AIML Department Scoring Above 80

```
SELECT S.Name, S.Department, M.Marks
FROM STUDENT S
JOIN MARKS M
ON S.StudentID = M.StudentID
WHERE S.Department = 'AIML'
AND M.Marks > 80;
```

| Name  | Department | Marks |
|-------|------------|-------|
| Durga | AIML       | 90    |
| Priya | AIML       | 88    |

  

| Name  | Marks |
|-------|-------|
| Rahul | 75    |
| Arjun | 65    |

**6. Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.**

**Step 1: Create STUDENT Table**

```
CREATE TABLE STUDENT (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(30),
    Department VARCHAR(20),
    Age INT,
    Marks INT
);
```

**Step 2: Insert Sample Data**

```
INSERT INTO STUDENT VALUES
(101,'Durga','AIML',20,90),
(102,'Rahul','CSE',21,75),
(103,'Priya','AIML',20,88),
(104,'Arjun','ECE',22,65);
```

**Query 1: Display Only Student Name and Department**

```
CREATE VIEW Student_View AS
SELECT Name, Department
FROM STUDENT;
SELECT * FROM Student_View;
```

Output:

| Name  | Department |
|-------|------------|
| Durga | AIML       |
| Rahul | CSE        |
| Priya | AIML       |
| Arjun | ECE        |

### Query 2: Display Students Scoring Above 80

```
CREATE VIEW Top_Students AS
SELECT Name, Marks
FROM STUDENT
WHERE Marks > 80;
SELECT * FROM Top_Students;
```

Output:

| Name  | Department |
|-------|------------|
| Durga | AIML       |
| Rahul | CSE        |
| Priya | AIML       |
| Arjun | ECE        |

  

| Name  | Marks |
|-------|-------|
| Durga | 90    |
| Priya | 88    |

### Query 3: Display AIML Students

```
CREATE VIEW AIML_Students AS
SELECT StudentID, Name, Marks
FROM STUDENT
WHERE Department = 'AIML';
SELECT * FROM AIML_Students;
```

Output:

| Name  | Department |
|-------|------------|
| Durga | AIML       |
| Rahul | CSE        |
| Priya | AIML       |
| Arjun | ECE        |

  

| Name  | Marks |
|-------|-------|
| Durga | 90    |
| Priya | 88    |

  

| StudentID | Name  | Marks |
|-----------|-------|-------|
| 101       | Durga | 90    |
| 103       | Priya | 88    |

**7. Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.**

**Relational Algebra:**

Retrieve names of students belonging to the AIML department:

$\pi \text{ Name } (\sigma \text{ DepartmentName} = \text{'AIML'}$   
 $(\text{STUDENT} \bowtie \text{STUDENT.DepartmentID} = \text{DEPARTMENT.DepartmentID DEPARTMENT}))$

Retrieve students who scored more than 80 marks:

$\pi \text{ Name} (\text{STUDENT} \bowtie \text{STUDENT.StudentID} = \text{MARKS.StudentID} (\sigma \text{ Marks} > 80 (\text{MARKS})))$

**8. Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements.**

**Create Tables**

```
CREATE TABLE Employees (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50),  
    DeptID INT,  
    Salary DECIMAL(10,2)  
);
```

**Departments Table**

```
CREATE TABLE Departments (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50)  
);
```

**Query 1: Replace Subquery with JOIN**

```
SELECT EmpName  
FROM Employees  
WHERE DeptID IN  
(  
    SELECT DeptID  
    FROM Departments  
    WHERE DeptName = 'IT'  
);
```

Output:

```
+-----+
| EmpName |
+-----+
| Charlie |
| Frank   |
| Ivy     |
+-----+
+-----+
| EmpName |
+-----+
| Charlie |
| Frank   |
| Ivy     |
+-----+
```

### Query 2: Add Index for Faster Searching

```
SELECT *
FROM Employees
WHERE Salary>60000;
```

```
+-----+
| EmpName |
+-----+
| Charlie |
| Frank   |
| Ivy     |
+-----+
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | Salary |
+-----+-----+-----+-----+
|      3 | Charlie |      103 | 70000.00 |
|      6 | Frank   |      103 | 75000.00 |
|      9 | Ivy     |      103 | 80000.00 |
+-----+-----+-----+-----+
```

### Query 3: Use JOIN Instead of Multiple Queries

```
SELECT EmpName,
(
SELECT DeptName
FROM Departments
WHERE Departments.DeptID=Employees.DeptID
) AS Department
FROM Employees;
```



Output:

| EmpID | EmpName | DeptID | Salary   |
|-------|---------|--------|----------|
| 3     | Charlie | 103    | 70000.00 |
| 6     | Frank   | 103    | 75000.00 |
| 9     | Ivy     | 103    | 80000.00 |

| EmpName | Department |
|---------|------------|
| Alice   | HR         |
| Bob     | Finance    |
| Charlie | IT         |
| David   | HR         |
| Emma    | Marketing  |
| Frank   | IT         |
| Grace   | Finance    |
| Henry   | Marketing  |
| Ivy     | IT         |
| Jack    | HR         |

#### Query 4: Create View for Frequently Used Query

CREATE VIEW EmployeeDetails AS

SELECT

E.EmpID,

E.EmpName,

D.DeptName,

E.Salary

FROM Employees E

JOIN Departments D

ON E.DeptID=D.DeptID;

SELECT \*

FROM EmployeeDetails;

Output:

| EmpName | DeptName  |
|---------|-----------|
| Alice   | HR        |
| Bob     | Finance   |
| Charlie | IT        |
| David   | HR        |
| Emma    | Marketing |
| Frank   | IT        |
| Grace   | Finance   |
| Henry   | Marketing |
| Ivy     | IT        |
| Jack    | HR        |

| EmpID | EmpName | DeptName  | Salary   |
|-------|---------|-----------|----------|
| 1     | Alice   | HR        | 45000.00 |
| 2     | Bob     | Finance   | 55000.00 |
| 3     | Charlie | IT        | 70000.00 |
| 4     | David   | HR        | 48000.00 |
| 5     | Emma    | Marketing | 60000.00 |
| 6     | Frank   | IT        | 75000.00 |
| 7     | Grace   | Finance   | 52000.00 |
| 8     | Henry   | Marketing | 58000.00 |
| 9     | Ivy     | IT        | 80000.00 |
| 10    | Jack    | HR        | 47000.00 |

### Query 5: Aggregate Query Optimization

```
SELECT DeptID,  
AVG(Salary)  
FROM Employees  
GROUP BY DeptID;
```

| EmpID | EmpName | DeptName  | Salary   |
|-------|---------|-----------|----------|
| 1     | Alice   | HR        | 45000.00 |
| 2     | Bob     | Finance   | 55000.00 |
| 3     | Charlie | IT        | 70000.00 |
| 4     | David   | HR        | 48000.00 |
| 5     | Emma    | Marketing | 60000.00 |
| 6     | Frank   | IT        | 75000.00 |
| 7     | Grace   | Finance   | 52000.00 |
| 8     | Henry   | Marketing | 58000.00 |
| 9     | Ivy     | IT        | 80000.00 |
| 10    | Jack    | HR        | 47000.00 |

  

| DeptID | AVG(Salary)  |
|--------|--------------|
| 101    | 46666.666667 |
| 102    | 53500.000000 |
| 103    | 75000.000000 |
| 104    | 59000.000000 |

**9. Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem.**

### Create Tables

```
CREATE TABLE Departments (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);
```

### Employees Table

```
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT,
    Salary DECIMAL(10,2),
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);
```

### Insert Sample Data

```
INSERT INTO Departments VALUES
(101,'HR'),
(102,'Finance'),
(103,'IT'),
(104,'Marketing');
```

```
INSERT INTO Employees VALUES
```

```
(1,'Alice',101,45000),  
(2,'Bob',102,55000),  
(3,'Charlie',103,70000),  
(4,'David',101,48000),  
(5,'Emma',104,60000),  
(6,'Frank',103,75000),  
(7,'Grace',102,52000),  
(8,'Henry',104,58000),  
(9,'Ivy',103,80000),  
(10,'Jack',101,47000);
```

### Query 1: Departments with Average Salary Greater Than Overall Average

```
SELECT  
    D.DeptName,  
    COUNT(E.EmpID) AS TotalEmployees,  
    SUM(E.Salary) AS TotalSalary,  
    AVG(E.Salary) AS AverageSalary  
FROM Employees E  
JOIN Departments D  
ON E.DeptID = D.DeptID  
GROUP BY D.DeptName  
HAVING AVG(E.Salary) >  
(  
    SELECT AVG(Salary)  
    FROM Employees  
);
```

Output:

| DeptName | TotalEmployees | TotalSalary | AverageSalary |
|----------|----------------|-------------|---------------|
| IT       | 3              | 225000.00   | 75000.000000  |

### Query 2: Highest Paid Employee in Each Department

```
SELECT  
    EmpName,  
    Salary,  
    DeptID  
FROM Employees E  
WHERE Salary =  
(  
    SELECT MAX(Salary)  
    FROM Employees  
    WHERE DeptID = E.DeptID  
);
```

Output:

| DeptName | TotalEmployees | TotalSalary | AverageSalary |
|----------|----------------|-------------|---------------|
| IT       | 3              | 225000.00   | 75000.000000  |

  

| EmpName | Salary   | DeptID |
|---------|----------|--------|
| Bob     | 55000.00 | 102    |
| David   | 48000.00 | 101    |
| Emma    | 60000.00 | 104    |
| Ivy     | 80000.00 | 103    |

### Query 3: Departments with More Than Two Employees

```
SELECT
    D.DeptName,
    COUNT(E.EmpID) AS EmployeeCount,
    AVG(E.Salary) AS AverageSalary
FROM Employees E
JOIN Departments D
ON E.DeptID = D.DeptID
GROUP BY D.DeptName
HAVING COUNT(E.EmpID) >
(
    SELECT AVG(EmployeeCount)
    FROM
    (
        SELECT COUNT(*) AS EmployeeCount
        FROM Employees
        GROUP BY DeptID
    ) AS DeptCounts
);
```

Output:

```
+-----+-----+-----+-----+
| DeptName | TotalEmployees | TotalSalary | AverageSalary |
+-----+-----+-----+-----+
| IT       |                3 | 225000.00 | 75000.000000 |
+-----+-----+-----+-----+

+-----+-----+-----+
| EmpName | Salary | DeptID |
+-----+-----+-----+
| Bob     | 55000.00 | 102 |
| David   | 48000.00 | 101 |
| Emma    | 60000.00 | 104 |
| Ivy     | 80000.00 | 103 |
+-----+-----+-----+

+-----+-----+-----+
| DeptName | EmployeeCount | AverageSalary |
+-----+-----+-----+
| HR       |                3 | 46666.666667 |
| IT       |                3 | 75000.000000 |
+-----+-----+-----+
```

**10. Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning.**

### Create Tables

```
CREATE TABLE Departments (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);
```

### Employees Table

```
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT,
    Salary DECIMAL(10,2),
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);
```

### Insert Sample Data

```
INSERT INTO Departments VALUES
(101,'HR'),
(102,'Finance'),
(103,'IT'),
(104,'Marketing');
INSERT INTO Employees VALUES
(1,'Alice',101,45000),
```

```
(2,'Bob',102,55000),
(3,'Charlie',103,70000),
(4,'David',101,48000),
(5,'Emma',104,60000),
(6,'Frank',103,75000),
(7,'Grace',102,52000),
(8,'Henry',104,58000),
(9,'Ivy',103,80000),
(10,'Jack',101,47000);
```

#### Query 1: Use JOIN

```
SELECT E.EmpName, D.DeptName
FROM Employees E
JOIN Departments D
ON E.DeptID = D.DeptID;
```

---

#### Output:

| EmpName | DeptName  |
|---------|-----------|
| Alice   | HR        |
| David   | HR        |
| Jack    | HR        |
| Bob     | Finance   |
| Grace   | Finance   |
| Charlie | IT        |
| Frank   | IT        |
| Ivy     | IT        |
| Emma    | Marketing |
| Henry   | Marketing |

#### Query 2: Use Subquery

```
SELECT EmpName, Salary
FROM Employees
WHERE Salary >
(
SELECT AVG(Salary)
FROM Employees
);
```

Output:

| EmpName | DeptName  |
|---------|-----------|
| Alice   | HR        |
| David   | HR        |
| Jack    | HR        |
| Bob     | Finance   |
| Grace   | Finance   |
| Charlie | IT        |
| Frank   | IT        |
| Ivy     | IT        |
| Emma    | Marketing |
| Henry   | Marketing |

  

| EmpName | Salary   |
|---------|----------|
| Charlie | 70000.00 |
| Emma    | 60000.00 |
| Frank   | 75000.00 |
| Ivy     | 80000.00 |

### Query 3: Use View

```
CREATE VIEW EmployeeDetails AS
SELECT
E.EmpID,
E.EmpName,
D.DeptName,
E.Salary
FROM Employees E
JOIN Departments D
ON E.DeptID = D.DeptID;
SELECT * FROM EmployeeDetails;
```



| EmpID | EmpName | DeptName  | Salary   |
|-------|---------|-----------|----------|
| 1     | Alice   | HR        | 45000.00 |
| 2     | Bob     | Finance   | 55000.00 |
| 3     | Charlie | IT        | 70000.00 |
| 4     | David   | HR        | 48000.00 |
| 5     | Emma    | Marketing | 60000.00 |
| 6     | Frank   | IT        | 75000.00 |
| 7     | Grace   | Finance   | 52000.00 |
| 8     | Henry   | Marketing | 58000.00 |
| 9     | Ivy     | IT        | 80000.00 |
| 10    | Jack    | HR        | 47000.00 |



**Code of Conduct:**

I CH. Durga Prasad (192425305) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

***Signature of the student:***

**RUBRICS FOR CALCULATION:**

| Criteria                  | Level 4<br>(Exemplary)                                                            | Level 3<br>(Proficient)                   | Level 2<br>(Developing)                    | Level 1<br>(Beginning)                        |
|---------------------------|-----------------------------------------------------------------------------------|-------------------------------------------|--------------------------------------------|-----------------------------------------------|
| Problem Understanding     | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps       | Partial understanding; misses key elements | Misinterprets or unable to understand problem |
| Method Selection          | Selects most appropriate method/formula with strong justification                 | Selects correct method with minor errors  | Inappropriate or partially correct method  | Unable to select method                       |
| Solution Process          | Logical, systematic steps with correct approach throughout                        | Mostly correct steps with minor mistakes  | Disorganized steps; multiple errors        | No clear process                              |
| Accuracy of Calculation   | All calculations accurate; correct final answer                                   | Minor calculation errors                  | Frequent errors; incorrect final answer    | No valid calculations                         |
| Interpretation of Results | Clearly interprets results with units and engineering relevance                   | Basic interpretation with minor omissions | Limited or unclear interpretation          | No interpretation                             |